

Example Code + LAB 3

Tuesday, May 26, 2026 15:51

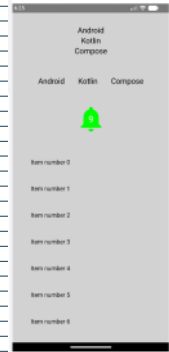
Jetpack Compose Basics:



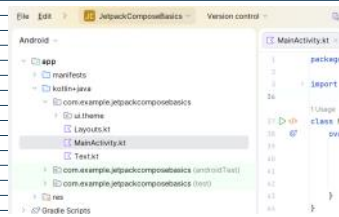
JetpackC...

- imported into IDE
- example code is from textbook (can see which chapter in package name)

```
1 package com.packt.chapterthree
2
3 > import ...
12
13 @Composable
```



Output of 'JetpackComposeBasics'
- can scroll vertically
- uses lazy column



```
1 package com.example.jetpackcomposebasics
2
3 import ...
36
37 class MainActivity : ComponentActivity() {
38     override fun onCreate(savedInstanceState: Bundle?) {
39         super.onCreate(savedInstanceState)
40         setContentView()
41         ExampleLayouts()
42     }
43 }
44
45 @Composable
46 fun ExampleLayouts() {
47     Column(
48         modifier = Modifier
49             .fillMaxSize()
50             .background(color = Color.LightGray)
51             .padding(all = 16.dp),
52         verticalArrangement = Arrangement.Center,
53         horizontalAlignment = Alignment.CenterHorizontally
54     ) {
55         SimpleColumn()
56         Spacer(modifier = Modifier.height(20.dp))
57         SimpleRow()
58         Spacer(modifier = Modifier.height(20.dp))
59         SimpleBox()
60         Spacer(modifier = Modifier.height(20.dp))
61         SimpleLazyColumn()
62         Spacer(modifier = Modifier.height(20.dp))
63         SimpleLazyRow()
64     }
65 }
66
67 @Composable
68 fun SimpleColumn() {
69     Column(
70         modifier = Modifier
71             .fillMaxSize()
72             .padding(all = 16.dp),
73         verticalArrangement = Arrangement.Center,
74         horizontalAlignment = Alignment.CenterHorizontally
75     ) {
76         Text(text = "Android", fontSize = 20.sp)
77         Text(text = "Kotlin", fontSize = 20.sp)
78         Text(text = "Compose", fontSize = 20.sp)
79     }
80 }
81
82 @Composable
83 fun SimpleRow() {
84     Row(
85         modifier = Modifier
86             .fillMaxSize()
87             .padding(all = 16.dp),
88         verticalAlignment = Alignment.CenterVertically,
89         horizontalArrangement = Arrangement.SpaceEvenly
90     ) {
91         Text(text = "Android", fontSize = 20.sp)
92         Text(text = "Kotlin", fontSize = 20.sp)
93         Text(text = "Compose", fontSize = 20.sp)
94     }
95 }
96
97 @Composable
98 fun SimpleBox() {
99     ...
100 }
```

```

82
83 ! Usage
84 @Composable
85 fun SimpleRow() {
86     Row(
87         modifier = Modifier
88             .fillMaxWidth()
89             .padding( all = 16.dp ),
90         verticalAlignment = Alignment.CenterVertically,
91         horizontalArrangement = Arrangement.SpaceEvenly
92     ) {
93         Text(text = "Android", fontSize = 20.sp)
94         Text(text = "Motlin", fontSize = 20.sp)
95         Text(text = "Compose", fontSize = 20.sp)
96     }
97 }
98
99 ! Usage
100 @Composable
101 fun SimpleBox() {
102     Box(
103         modifier = Modifier
104             .size( size = 100.dp )
105             .padding( all = 8.dp ),
106         contentAlignment = Alignment.Center
107     ) {
108         Icon(
109             imageVector = Icons.Filled.Notifications,
110             contentDescription = null,
111             tint = Color.Green,
112             modifier = Modifier.size( size = 80.dp )
113         )
114         Text(text = "99", color = Color.White, fontSize = 24.sp, textAlign = TextAlign.Center)
115     }
116 }
117
118 ! Usage
119 @Composable
120 fun SimpleLazyColumn() {
121     var clickedItem by remember { mutableStateOf( value = -1 ) } // State to track the last clicked item
122
123     LazyColumn(
124         modifier = Modifier
125             .fillMaxWidth()
126             .background( color = LightGray )
127             .padding( all = 8.dp )
128     ) {
129         items( count = 100 ) { index ->
130             Text(
131                 text = if ( clickedItem == index ) "Clicked: Item number $index" else "Item number $index",
132                 modifier = Modifier
133                     .fillMaxWidth()
134                     .padding( all = 8.dp )
135                     .clickable { clickedItem = index } // Update clickedItem on click
136                     .background( color = if ( clickedItem == index ) Color.Transparent else Color.Transparent )
137                     .padding( all = 16.dp ),
138                 fontSize = 16.sp
139             )
140         }
141     }
142 }
143
144 ! Usage
145 @Composable
146 fun SimpleLazyRow() {
147     LazyRow(
148         modifier = Modifier
149             .fillMaxWidth()
150             .background( color = LightGray )
151             .padding( all = 8.dp )
152     ) {
153         items( count = 100 ) { index ->
154             Text(
155                 text = "Item number $index",
156                 modifier = Modifier.padding( all = 8.dp ),
157                 fontSize = 16.sp
158             )
159         }
160     }
161 }

```

Week 3 Layout App:

Week3La

-02:10

Week 3: Lab Exercise – Simple Calculator

```

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContentView {
            MaterialTheme {
                CalculatorScreen()
            }
        }
    }
}

@Composable
fun CalculatorScreen() {
    var number1 by remember { mutableStateOf("") }
    var number2 by remember { mutableStateOf("") }
    var result by remember { mutableStateOf(0) }

    Surface(
        modifier = Modifier.fillMaxSize()
    ) {
        Column(
            modifier = Modifier
                .fillMaxSize()
                .padding(24.dp),

            verticalArrangement = Arrangement.Center,
            horizontalAlignment = Alignment.CenterHorizontally
        ) {

```

package com.example.week3labexercise_simplecalculator

```

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.text.KeyboardOptions
import androidx.compose.material3.Button
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.input.KeyboardType
import androidx.compose.ui.tooling.preview.Preview
import androidx.compose.ui.unit.dp
import com.example.week3labexercise_simplecalculator.ui.theme.Week3LabExerciseSimpleCalculatorTheme

```

*ONE big composable function

Asked Prof what best practice is:

- depends on project requirements
- modularity is always preferred
- but in simple examples like this, it's easier to just use one
- no definitive answer, but modularity is good!

```

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            MaterialTheme {
                CalculatorScreen()
            }
        }
    }
}

@Composable
fun CalculatorScreen() {
    var number1 by remember { mutableStateOf("") }
    var number2 by remember { mutableStateOf("") }
    var result by remember { mutableStateOf(0) }

    Surface(
        modifier = Modifier.fillMaxSize()
    ) {
        Column(
            modifier = Modifier
                .fillMaxSize()
                .padding(24.dp),
            verticalArrangement = Arrangement.Center,
            horizontalAlignment = Alignment.CenterHorizontally
        ) {
            Text(
                text = "Simple Calculator",
                style = MaterialTheme.typography.headlineMedium
            )
            Spacer(modifier = Modifier.height(24.dp))
            OutlinedTextField(

```

```

verticalArrangement = Arrangement.Center,
horizontalAlignment = Alignment.CenterHorizontally
){

    Text(
        text = "Simple Calculator",
        style = MaterialTheme.typography.headlineMedium
    )

    Spacer(modifier = Modifier.height(24.dp))

    OutlinedTextField(
        value = number1,
        onChange = { number1 = it },
        label = { Text("First Number") },
        keyboardOptions = KeyboardOptions(
            keyboardType = KeyboardType.Number
        ),
        modifier = Modifier.fillMaxWidth()
    )

    Spacer(modifier = Modifier.height(16.dp))

    OutlinedTextField(
        value = number2,
        onChange = { number2 = it },
        label = { Text("Second Number") },
        keyboardOptions = KeyboardOptions(

```

```

            keyboardType = KeyboardType.Number
        ),
        modifier = Modifier.fillMaxWidth()
    )

    Spacer(modifier = Modifier.height(24.dp))

    Row(
        horizontalArrangement = Arrangement.spacedBy(16.dp)
    ){

```

```

        Button(
            onClick = {
                val n1 = number1.toIntOrNull() ?: 0
                val n2 = number2.toIntOrNull() ?: 0
                result = n1 + n2
            }
        ){
            Text("Add")
        }

        Button(
            onClick = {
                val n1 = number1.toIntOrNull() ?: 0
                val n2 = number2.toIntOrNull() ?: 0
                result = n1 - n2
            }
        ){
            Text("Subtract")
        }

        Spacer(modifier = Modifier.height(24.dp))

        Text(
            text = "Result: $result",
            style = MaterialTheme.typography.headlineSmall
        )
    }
}

```

```

    }
){
    Text("Subtract")
}
}

Spacer(modifier = Modifier.height(24.dp))

Text(
    text = "Result: $result",
    style = MaterialTheme.typography.headlineSmall
)
}
}

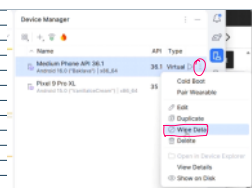
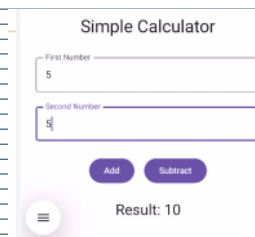
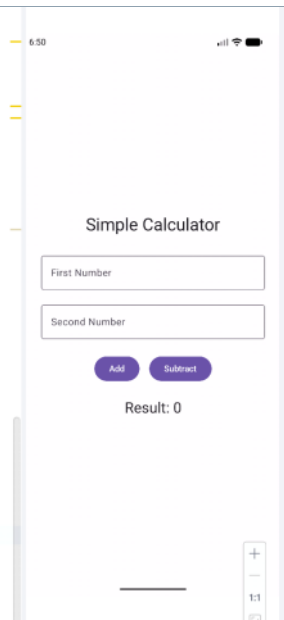
```

```

){
    Text(
        text = "Simple Calculator",
        style = MaterialTheme.typography.headlineMedium
    )
    Spacer(modifier = Modifier.height(24.dp))
    OutlinedTextField(
        value = number1,
        onChange = { number1 = it },
        label = { Text("First Number") },
        keyboardOptions = KeyboardOptions(
            keyboardType = KeyboardType.Number
        ),
        modifier = Modifier.fillMaxWidth()
    )
    Spacer(modifier = Modifier.height(16.dp))
    OutlinedTextField(
        value = number2,
        onChange = { number2 = it },
        label = { Text("Second Number") },
        keyboardOptions = KeyboardOptions(
            keyboardType = KeyboardType.Number
        ),
        modifier = Modifier.fillMaxWidth()
    )
    Spacer(modifier = Modifier.height(24.dp))
    Row(
        horizontalArrangement = Arrangement.spacedBy(16.dp)
    ){
        Button(
            onClick = {
                val n1 = number1.toIntOrNull() ?: 0
                val n2 = number2.toIntOrNull() ?: 0
                result = n1 + n2
            }
        ){
            Text("Add")
        }
        Button(
            onClick = {
                val n1 = number1.toIntOrNull() ?: 0
                val n2 = number2.toIntOrNull() ?: 0
                result = n1 - n2
            }
        ){
            Text("Subtract")
        }
    }
    Spacer(modifier = Modifier.height(24.dp))
    Text(
        text = "Result: $result",
        style = MaterialTheme.typography.headlineSmall
    )
}
}
}

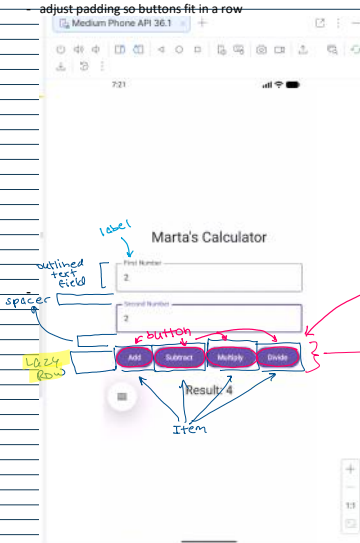
```

** from text field
 is a string, need
 to convert to Int,
 then can use addition*



- if having trouble with emulators, DON'T DELETE, just 'Wipe Data'
 - deleting might mess up past configurations (for some reason)

// added multiply + divide buttons & functionality
 - added a lazy row and put all buttons in their own 'item' wrapper within
 - adjust padding so buttons fit in a row



NOTE: variables
 on click the variable number 1 & number 2 are reassigned to whatever number is inputted in the text field. the variable number 1 is updated and the output is calculated and the output is displayed in the result field.

- added 2 buttons and put into a LazyRow
 had to make 2 changes:
1. add lazy row, then subclasses 'items' and out each button into its own item container
 2. adjust modifier for LazyRow to 'fillMaxWidth' - remove any unnecessary spacing

```

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            MaterialTheme {
                CalculatorScreen()
            }
        }
    }
}

@Composable
fun CalculatorScreen() {
    var number1 by remember { mutableStateOf(value = "") }
    var number2 by remember { mutableStateOf(value = "") }
    var result by remember { mutableIntStateOf(value = 0) }

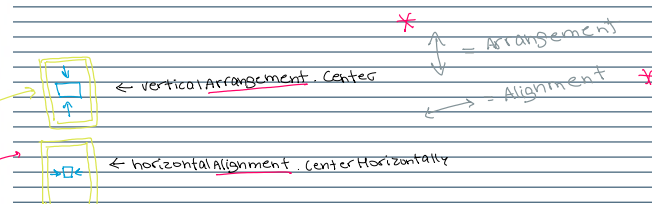
    Surface(
        modifier = Modifier.fillMaxSize()
    ) {
        Column(
            modifier = Modifier
                .fillMaxSize()
                .padding(all = 24.dp),
            verticalArrangement = Arrangement.Center,
            horizontalAlignment = Alignment.CenterHorizontally
        ) {
            Text(
                text = "Marta's Calculator",
            )
        }
    }
}
  
```

upon initialization the state variables have default values: empty or 0
 states to store values of inputs & output

main column, holds input/output fields & buttons

padding b/w content & edge of screen
 vertically, content is centered

} PAGE TITLE



```

57 verticalArrangement = Arrangement.Center,
58 horizontalAlignment = Alignment.CenterHorizontally
59
60 } {
61     Text(
62         text = "Marta's Calculator",
63         style = MaterialTheme.typography.headlineMedium
64     )
65 }
66
67 Spacer(modifier = Modifier.height( height = 24.dp))
68 OutlinedTextField(
69     value = number1,
70     onValueChange = { number1 = it },
71     label = { Text( text = "First Number" ) },
72     keyboardOptions = KeyboardOptions(
73         keyboardType = KeyboardType.Number
74     ),
75     modifier = Modifier.fillMaxWidth()
76 )
77
78 Spacer(modifier = Modifier.height( height = 16.dp))
79 OutlinedTextField(
80     value = number2,
81     onValueChange = { number2 = it },
82     label = { Text( text = "Second Number" ) },
83     keyboardOptions = KeyboardOptions(
84         keyboardType = KeyboardType.Number
85     ),
86     modifier = Modifier.fillMaxWidth()
87 )
88
89 Spacer(modifier = Modifier.height( height = 24.dp))
90 LazyRow(
91     modifier = Modifier.fillMaxWidth(), // fillMaxWidth would make it take full height.
92     verticalAlignment = Alignment.CenterVertically
93 ) {
94     // add button
95     item {
96         Button(
97             onClick = {
98                 val n1 = number1.toIntOrNull() ?: 0
99                 val n2 = number2.toIntOrNull() ?: 0
100                 result = n1 + n2
101             },
102             Text( text = "Add" )
103         )
104     }
105     // subtract button
106     item {
107         Button(
108             onClick = {
109                 val n1 = number1.toIntOrNull() ?: 0
110                 val n2 = number2.toIntOrNull() ?: 0
111                 result = n1 - n2
112             },
113             Text( text = "Subtract" )
114         )
115     }
116
117     // multiply button
118     item {
119         Button(
120             onClick = {
121                 val n1 = number1.toIntOrNull() ?: 0
122                 val n2 = number2.toIntOrNull() ?: 0
123                 result = n1 * n2
124             },
125             Text( text = "Multiply" )
126         )
127     }
128
129     // divide button
130     item {
131         Button(
132             onClick = {
133                 val n1 = number1.toIntOrNull() ?: 0
134                 val n2 = number2.toIntOrNull() ?: 0
135                 result = if (n2 != 0) n1 / n2 else 0
136             },
137             Text( text = "Divide" )
138         )
139     }
140 } // LazyRow
141
142 Spacer(modifier = Modifier.height( height = 24.dp))
143 Text(
144     text = "Result: $result",
145     style = MaterialTheme.typography.headlineSmall
146 )
147
148 }
149
150 }
151
152 }

```

verticalArrangement = Arrangement.Center, horizontalAlignment = Alignment.CenterHorizontally

horizontalAlignment: CenterHorizontally

PAGE TITLE

spacers b/w each object

user input is stored in state var

state variable

text field headings

same here the value of the var is reassigned

explicitly state input type so it is stored as an Int & not a string

make field as wide as column

made all content group together at center

LAZY ROW

each object = item {}

BUTTON 1

MATH. answer is assigned to var. result

BUTTON 2

BUTTON 3

BUTTON 4

item {} object

Button() {}

onClick - events - formatting of button

what's displayed on UI (text)

variables in output text: "4" "var"

it is then updated live in this Text object.